UNIVERSIDADE DO ESTADO DO RIO GRANDE DO NORTE (UERN)

Disciplina: Sistemas Operacionais

Professora: Artemísia Kimberlly

Atividade: Projeto Prático de Concorrência e Sincronização

1. Objetivo do Projeto

Este trabalho prático tem como objetivo consolidar os conceitos de Seção Crítica, Condição de Corrida, Deadlock e Inanição (Starvation). Vocês deverão implementar algoritmos clássicos de sincronização de processos e threads, contrastando as vantagens e limitações de diferentes mecanismos estruturais: Semáforos, Mutexes e Monitores.

2. Distribuição dos Temas

A turma (24 alunos) será dividida em 8 trios. Cada trio ficará responsável por um problema clássico e um mecanismo de sincronização específico.

Trio	O Problema Clássico	Mecanismo Exigido	Linguagem Sugerida
Trio 01 - Jean Marota, Ferdinando e Jorge	Produtor-Consumidor	Semáforos	C / C++ (POSIX)
Trio 02 - Mateus Neri, Guilherme sousa e junior	Produtor-Consumidor	Monitores	Java / C# / Python
Trio 03 - Micael, Rafael e Manuel	Leitores-Escritores	Semáforos	C / C++ (POSIX)
Trio 04 - Sayan, Geovane e Antoniel	Leitores-Escritores	Mutexes	C / C++ (POSIX)

Trio 05 - Viviane, Guilherme Holanda e Rafael rocha	Jantar dos Filósofos	Semáforos	C / C++ (POSIX)
Trio 06 - Clara, Jayane e Jaelson	Jantar dos Filósofos	Mutexes	C / C++ (POSIX)
Trio 07 -	Barbeiro Dorminhoco	Semáforos	C / C++ (POSIX)

3. Especificações Técnicas de Implementação

Para que a implementação seja validada, o código de todos os grupos deve obedecer aos seguintes requisitos obrigatórios:

1. **Simulação de Tempo real:** Vocês devem utilizar funções de atraso (sleep(), usleep(), Thread.sleep()) dentro e fora das seções críticas para simular o "tempo de processamento". Isso forçará o escalonador do SO a realizar trocas de contexto (*context switches*) e provará que a sincronização funciona.
2. **Console/Log Descritivo:** O terminal deve imprimir o estado do sistema de forma clara e legível.
    - *Exemplo aceitável:* [+] Produtor 1 inseriu item. (Espaços livres: 4)
    - *Exemplo aceitável:* [!] Filósofo 3 tentando pegar garfo esquerdo...
3. **Linguagens e Bibliotecas:**
    - Para **Semáforos e Mutexes:** Recomenda-se C ou C++ utilizando a biblioteca pthreads (POSIX Threads) e semaphore.h.
    - Para **Monitores:** Recomenda-se linguagens Orientadas a Objetos. A sincronização deve ser feita utilizando os recursos nativos de encapsulamento da linguagem (ex: synchronized no Java, lock no C#, ou Condition no Python). O programador *não* deve usar chamadas explícitas de semáforo da biblioteca padrão nesses casos.
4. **Tratamento de Fim de Execução:** O programa não precisa rodar para sempre, mas deve executar loops suficientes (ex: 50 interações por thread) para demonstrar que não há *Deadlocks* ou *Starvation*.

4. Entregáveis do Projeto

Cada trio deverá realizar **três entregas**:

A. Repositório de Código (Link do GitHub)

- Código-fonte completo, limpo e devidamente comentado.

- Um arquivo README.md contendo:
    - Nome dos integrantes.
    - Instruções exatas de como compilar e executar o código localmente.
    - Breve descrição da estratégia utilizada.

B. Vídeo de Demonstração (3 a 5 minutos)

- **Demonstração Visual:** Compartilhamento de tela mostrando a execução do programa no terminal, provando pelo log que as regras do problema foram respeitadas.
- **O Teste do Caos:** No vídeo, vocês devem comentar a linha de código responsável pela trava (desativar o Mutex/Semáforo/Monitor), recompilar e mostrar o programa "quebrando" ao vivo (gerando resultados inconsistentes ou travando).

C. Apresentação em Sala de Aula (8 a 10 minutos por Trio)

- Vocês não devem mostrar código fonte de baixo nível no projetor.
- A apresentação deve focar na **LÓGICA** utilizem diagramas de blocos, fluxogramas ou animações simples para explicar para a turma *como* o mecanismo designado resolveu os gargalos do problema.

5. Rubrica de Avaliação

O projeto valerá **3 pontos**, distribuídos da seguinte forma:

- **Lógica de Sincronização e Código (40%):** O mecanismo exigido foi implementado corretamente? Evitou *busy waiting*, *deadlocks* e inanição (*starvation*)? O código está documentado?
- **Qualidade do Vídeo de Demonstração (20%):** O log do terminal permitiu acompanhar a execução claramente? O teste do caos foi demonstrado?
- **Apresentação e Defesa em Sala (30%):** O trio conseguiu traduzir o funcionamento do código para os colegas usando recursos visuais adequados? O grupo demonstrou domínio teórico ao ser questionado?
- **Boas Práticas de Engenharia (10%):** Repositório organizado, README.md claro e instruções de compilação precisas.

6. Prazos

- **Data limite para envio do repositório e vídeo (Sistema):** 8 de Outubro, 23h59
- **Apresentações em Sala:** 9 de Outubro, em sala de aula, não havendo possibilidade de ser postergado devido cronograma.

Boa sorte e bom trabalho! :D